

PROJECT 1: UNCONSTRAINED OPTIMIZATION

OPTIMIZATION METHODS, YACHAY TECH UNIVERSITY

Kevin Sanchez
kelth.joss53@gmail.com

Sunday 20th September, 2026

1 Introduction

Unconstrained optimization deals with problems of the form

$$\min_{x \in \mathbb{R}^n} f(x), \quad (1)$$

where no restriction is imposed on the decision variable x , as opposed to constrained optimization, where x must additionally satisfy a set of equality and/or inequality conditions. When f is differentiable, a point x^* can only be a local minimizer if it satisfies the first-order necessary condition $\nabla f(x^*) = 0$, and it is confirmed to be a strict local minimizer if, in addition, the Hessian $\nabla^2 f(x^*)$ is positive definite (second-order sufficiency condition). These two conditions are the theoretical backbone of both the analytical study of stationary points and the iterative numerical methods addressed in this work.

Most unconstrained optimization methods are iterative: starting from an initial guess x_0 , they generate a sequence $\{x_k\}$ that is expected to converge to a point satisfying $\nabla f(x^*) = 0$. They differ in how much local information about f they use to decide, at each step, in which direction to move and how far. In this project, three classical iterative methods are implemented and compared: the Steepest Descent method, which relies only on first-order information (the gradient); Newton's Method,

which additionally uses second-order information (the Hessian); and the Conjugate Gradient method, which improves on Steepest Descent using only first-order information by reusing information from previous iterations.

The objectives of this work are: (i) to implement the Steepest Descent, Newton, and Conjugate Gradient methods for unconstrained minimization; (ii) to apply them to a given test function and compare their convergence behavior, both theoretically and experimentally; and (iii) to determine analytically the stationary points of two additional functions and classify them using the second-order sufficiency conditions, corroborating the results numerically.

2 Methods

All three methods considered in this work share the same general iterative scheme:

$$x_{k+1} = x_k + \alpha_k d_k, \quad (2)$$

where d_k is a *descent direction* and $\alpha_k > 0$ is a *step size*. They differ in how d_k is computed; the step size α_k , in all three implementations, is chosen by the same backtracking line search, so that any difference in performance can be attributed to the choice of direction rather than to the step size rule.

⁰Code available at https://github.com/KevJoss/Optimization_methods_course

2.1 Backtracking line search

Given a descent direction d_k (one satisfying $\nabla f(x_k)^T d_k < 0$), the step size α_k is chosen to satisfy the Armijo sufficient-decrease condition:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^T d_k, \quad (3)$$

with $c_1 \in (0, 1)$. Starting from $\alpha_k = 1$, the step is halved until Equation 3 holds. This guarantees that f decreases at every iteration, regardless of how far x_0 is from the solution.

2.2 Steepest Descent

The Steepest Descent method uses $d_k = -\nabla f(x_k)$, the direction of local maximum decrease of f at x_k .

Algorithm 1 Steepest Descent

```

1: Given  $x_0$ , tolerance  $\varepsilon$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   if  $\|\nabla f(x_k)\| < \varepsilon$  then
4:     stop, return  $x_k$ 
5:   end if
6:    $d_k \leftarrow -\nabla f(x_k)$ 
7:   choose  $\alpha_k$  by backtracking line search
8:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
9: end for
```

Its main weakness is that it only uses local, first-order information: when f has narrow, elongated valleys (i.e., when $\nabla^2 f$ is ill-conditioned near the solution), consecutive gradients point in sharply different directions, producing a characteristic *zig-zagging* behavior. As a result, Steepest Descent converges only *linearly*: $\|x_{k+1} - x^*\| \leq \rho \|x_k - x^*\|$ for some $\rho \in (0, 1)$ depending on the condition number of $\nabla^2 f(x^*)$, which can make convergence very slow in practice.

2.3 Newton's Method

Newton's Method uses both the gradient and the Hessian of f . At each iterate x_k , it builds the local quadratic approximation of f and

moves to its minimizer, i.e., d_k solves the linear system

$$\nabla^2 f(x_k) d_k = -\nabla f(x_k). \quad (4)$$

If $\nabla^2 f(x^*)$ is positive definite (nonsingular) at the solution, Newton's Method converges *quadratically* in a neighborhood of x^* : $\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$, which in practice means the number of correct digits roughly doubles at every iteration. This is considerably faster than the linear convergence of Steepest Descent, at the cost of requiring the Hessian (and solving a linear system) at every step. If $\nabla^2 f(x^*)$ is singular, however, this quadratic guarantee no longer holds, and the observed convergence rate degrades — an effect discussed with the experimental results.

Algorithm 2 Newton's Method (damped)

```

1: Given  $x_0$ , tolerance  $\varepsilon$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   if  $\|\nabla f(x_k)\| < \varepsilon$  then
4:     stop, return  $x_k$ 
5:   end if
6:   solve  $\nabla^2 f(x_k) d_k = -\nabla f(x_k)$ 
7:   choose  $\alpha_k$  by backtracking line search
8:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
9: end for
```

2.4 Conjugate Gradient Method

The Conjugate Gradient method is a compromise between the two previous methods: it does not require the Hessian, but it improves on Steepest Descent by reusing information from the previous direction instead of discarding it. The (nonlinear, Fletcher–Reeves) direction is

$$d_{k+1} = -\nabla f(x_{k+1}) + \beta_{k+1} d_k, \quad \beta_{k+1} = \frac{\|\nabla f(x_{k+1})\|^2}{\|\nabla f(x_k)\|^2}, \quad (5)$$

with $d_0 = -\nabla f(x_0)$.

For an exactly quadratic f with exact line search, this direction is conjugate with respect to $\nabla^2 f$, and the method converges in at most n iterations (n = number of variables). For general (non-quadratic) f , this finite-step property is lost, so the direction is periodically

restarted to plain steepest descent (every n iterations) to keep the method well behaved. In practice, its convergence is faster than Steepest Descent but generally slower than Newton's Method, since it does not use curvature information.

Algorithm 3 Conjugate Gradient (Fletcher–Reeves)

```

1: Given  $x_0$ , tolerance  $\varepsilon$ ;  $g_0 \leftarrow \nabla f(x_0)$ ,  $d_0 \leftarrow -g_0$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   if  $\|g_k\| < \varepsilon$  then
4:     stop, return  $x_k$ 
5:   end if
6:   choose  $\alpha_k$  by backtracking line search along  $d_k$ 
7:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ ;  $g_{k+1} \leftarrow \nabla f(x_{k+1})$ 
8:   if  $(k+1) \bmod n = 0$  then
9:      $\beta_{k+1} \leftarrow 0$  {restart}
10:  else
11:     $\beta_{k+1} \leftarrow \|g_{k+1}\|^2 / \|g_k\|^2$ 
12:  end if
13:   $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$ 
14: end for

```

3 Experimental Environment

3.1 Personal Parameter

As specified in the assignment, d is obtained from the last digit of the student ID plus 2. For this project, $d = 3$.

3.2 Part I: Test Function

The function used to compare the three methods is

$$f(x_1, x_2) = (x_1 - d)^4 + (x_1 - 2dx_2)^2, \quad (6)$$

which, for $d = 3$, becomes $f(x_1, x_2) = (x_1 - 3)^4 + (x_1 - 6x_2)^2$. All three methods are applied from the same initial point $x_0 = [1, 3]^T$, as specified in the assignment. Since f is a sum of two even-power (hence non-negative) terms, its global minimum value is $f^* = 0$, attained at $x^* = (d, 0.5) = (3, 0.5)$, the only

point where both terms vanish simultaneously. This closed-form solution is used as ground truth: besides the gradient norm $\|\nabla f(x_k)\|$ tabulated for the first 10 iterations of each method, the distance $\|x_k - x^*\|$ is used to plot iteration-vs-error curves and to report each method's final accuracy.

3.3 Part II: Test Functions

The two functions studied in Part II, with $d = 3$, are

$$f_a(x_1, x_2) = 3x_1 + \frac{100}{x_1 x_2} + 3x_2, \quad (7)$$

$$f_b(x_1, x_2) = (x_1 - 3)^2 + x_1 x_2 + (x_2 - 3)^2. \quad (8)$$

Their stationary points are first determined analytically (Section 4) and then corroborated numerically by (i) evaluating the analytical gradient and Hessian at the computed point, and (ii) running Newton's Method, starting from $x_0 = (4, 2)$ for f_a and $x_0 = (0, 0)$ for f_b , to confirm convergence to the same point.

3.4 Algorithm Settings

Table 1 summarizes the parameters shared by all three iterative methods.

Table 1: Common algorithm settings.

Parameter	Value
Stopping tolerance ε	10^{-6}
Line search initial step α_0	1
Line search shrink factor ρ	0.5
Armijo constant c_1	10^{-4}
Max. iterations (Steepest Descent)	200,000
Max. iterations (Newton)	100
Max. iterations (Conjugate Gradient)	200,000

The much larger iteration budget allotted to Steepest Descent and Conjugate Gradient (compared to Newton) reflects their theoretically slower, linear convergence rate on this particular function, discussed in Section 4.

3.5 Software

All methods were implemented in Python 3, using NumPy for linear algebra operations (e.g., solving the linear system in Newton’s Method) and pandas/Matplotlib for post-processing and plotting the experimental results.

4 Experimental Results

4.1 Part I: Iteration Tables

Tables 3, 4, and 5 (Appendix A) show the first 10 iterations of Steepest Descent, Newton’s Method, and Conjugate Gradient, respectively, all starting from $x_0 = [1, 3]^T$.

4.2 Part I: Convergence Plot

Figure 1 shows $\|x_k - x^*\|$ against the iteration number (log-log scale) for all three methods over their full run. Steepest Descent appears as a straight line, the signature of linear convergence with a constant rate. Conjugate Gradient shows a visible jump at the iterations where the direction is restarted, after which it also decreases roughly linearly, but faster than Steepest Descent. Newton’s Method drops sharply and finishes in only 16 iterations; however, its rate is, in this particular case, also linear rather than quadratic (with an observed constant ratio of exactly $2/3$ between consecutive errors), because $\nabla^2 f(x^*)$ turns out to be singular, so the classical quadratic-convergence guarantee stated in Section 2 does not strictly hold here.

Table 2 summarizes the number of iterations each method needed to reach $\|\nabla f(x_k)\| < 10^{-6}$.

Table 2: Iterations required to converge.

Method	Iterations	Final $\ x_k - x^*\ $
Steepest Descent	65,917	6.26×10^{-3}
Newton	16	4.63×10^{-3}
Conjugate Gradient	12,341	6.29×10^{-3}

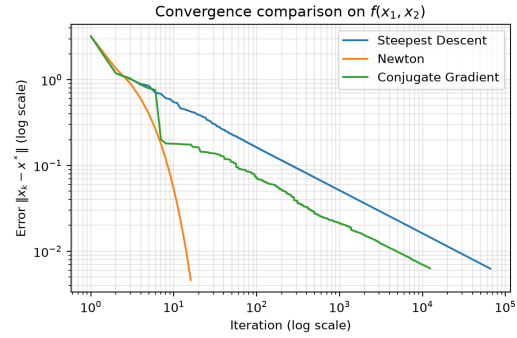


Figure 1: Convergence comparison of the three methods on $f(x_1, x_2)$, from $x_0 = [1, 3]^T$ to $x^* = (3, 0.5)$.

4.3 Part II: Stationary Points

Table 6 (Appendix A) summarizes the stationary points found analytically for the two functions of Part II (with $d = 3$), together with their classification from the second-order sufficiency conditions.

Both results were corroborated numerically: evaluating the analytical gradient at each x^* gives $\nabla f(x^*) \approx (0, 0)$ (up to floating-point precision), and running Newton’s Method from a nearby point converges exactly to the same x^* in both cases (7 iterations for f_a from $x_0 = (4, 2)$; a single Newton step for f_b from $x_0 = (0, 0)$, as expected since f_b is exactly quadratic).

5 Conclusions

5.1 Steepest Descent

Steepest Descent did converge toward the expected exact solution $x^* = (3, 0.5)$, reaching a final error $\|x_k - x^*\| \approx 6.26 \times 10^{-3}$ after satisfying the gradient tolerance. However, it required 65,917 iterations to do so — orders of magnitude more than the other two methods. This matches the theoretical expectation exactly: since it only uses first-order information, its convergence rate is linear, and the elongated, ill-conditioned shape of f near x^* (caused by the quartic term $(x_1 - d)^4$) makes that linear rate very slow in practice, with visi-

ble zig-zagging in the gradient norm from one iteration to the next. In practice, this method is reliable but impractical whenever high precision is required in reasonable time.

5.2 Newton's Method

Newton's Method reached the exact solution in only 16 iterations, by far the fastest of the three. However, the *rate* at which it converged did not match the classical expectation: textbook theory predicts quadratic convergence near x^* , but the experimental error decreased with a constant ratio of exactly $2/3$ between consecutive iterations — linear, not quadratic. This discrepancy between the expected and observed behavior is fully explained by the fact that $\nabla^2 f(x^*)$ is singular (positive semi-definite, not positive definite) for this particular function, which violates the nonsingularity assumption required for the quadratic-convergence guarantee. This illustrates an important, often overlooked point: Newton's superior practical speed does not automatically imply its best-known theoretical convergence order, and the two should be checked independently.

5.3 Conjugate Gradient

Conjugate Gradient converged to x^* in 12,341 iterations: more than five times faster than Steepest Descent, without ever requiring the Hessian. This matches the expected role of CG as an intermediate method between Steepest Descent and Newton. The periodic restart (every $n = 2$ iterations) is visible in the convergence plot as sharp drops in the error, after which the method resumes a roughly linear decrease, faster than plain Steepest Descent but not as fast as Newton, exactly as anticipated given it only reuses first-order information from previous steps.

5.4 Part II

The stationary points obtained analytically for f_a and f_b were both confirmed numerically: the gradient evaluated at each point was

approximately zero, the sign of the Hessian eigenvalues matched the predicted classification (local minimum in both cases), and Newton's Method, started near each point, converged exactly to it. This agreement between the hand-derived and the numerically verified results supports the correctness of both the analytical work and the implementation.

5.5 Recommendation

Based on these results, Newton's Method is the recommended choice whenever the Hessian is available and reasonably cheap to compute (or to factorize), as it reached the solution with dramatically fewer iterations, even in this case where its quadratic guarantee did not strictly hold. When the Hessian is unavailable or too expensive, Conjugate Gradient is a much better alternative than plain Steepest Descent, since it offers a substantial speedup at essentially the same per-iteration cost (only reusing the previous direction). Steepest Descent should mainly be regarded as a simple baseline method, useful for validation and teaching purposes, but generally impractical on its own for problems that require high accuracy.

A Detailed Iteration Tables and Classification

Table 3: Steepest Descent: first 10 iterations.

k	x_1	x_2	$f(x_k)$	$\ \nabla f(x_k)\ $
0	1.0000	3.0000	3.0500e+02	2.1441e+02
1	2.0312	-0.1875	1.0843e+01	3.7969e+01
2	1.9894	0.4043	1.2333e+00	7.2405e+00
3	2.1457	0.2407	1.0250e+00	8.4907e+00
4	2.1628	0.3722	4.9638e-01	2.6295e+00
5	2.3183	0.3192	3.7829e-01	4.8570e+00
6	2.3255	0.3948	2.0882e-01	1.4121e+00
7	2.4076	0.3624	1.7750e-01	2.8216e+00
8	2.4133	0.4061	1.1901e-01	8.9966e-01
9	2.4667	0.3885	9.9210e-02	1.6596e+00

Table 4: Newton's Method: first 10 iterations.

k	x_1	x_2	$f(x_k)$	$\ \nabla f(x_k)\ $
0	1.0000	3.0000	3.0500e+02	2.1441e+02
1	1.6667	0.2778	3.1605e+00	9.4815e+00
2	2.1111	0.3519	6.2430e-01	2.8093e+00
3	2.4074	0.4012	1.2332e-01	8.3239e-01
4	2.6049	0.4342	2.4359e-02	2.4664e-01
5	2.7366	0.4561	4.8117e-03	7.3077e-02
6	2.8244	0.4707	9.5045e-04	2.1652e-02
7	2.8829	0.4805	1.8774e-04	6.4155e-03
8	2.9220	0.4870	3.7085e-05	1.9009e-03
9	2.9480	0.4913	7.3255e-06	5.6323e-04

Table 5: Conjugate Gradient (Fletcher-Reeves): first 10 iterations.

k	x_1	x_2	$f(x_k)$	$\ \nabla f(x_k)\ $
0	1.0000	3.0000	3.0500e+02	2.1441e+02
1	2.0312	-0.1875	1.0843e+01	3.7969e+01
2	2.0123	0.7962	8.5955e+00	3.4478e+01
3	2.1589	0.2778	7.4269e-01	6.0691e+00
4	2.2116	0.4302	5.2308e-01	5.1944e+00
5	2.2538	0.3609	3.1785e-01	1.8251e+00
6	3.1630	0.6174	2.9391e-01	6.5845e+00
7	3.1797	0.5159	8.1545e-03	1.0301e+00
8	3.1745	0.5425	7.4532e-03	9.7953e-01
9	3.1767	0.5274	1.1255e-03	1.5469e-01

Table 6: Part II: stationary points and their classification.

Function	Stationary point x^*	Hessian eigenvalues	Classification
$f_a = 3x_1 + \frac{100}{x_1 x_2} + 3x_2$	(3.2183, 3.2183)	(0.9322, 2.7965)	Local minimum
$f_b = (x_1 - 3)^2 + x_1 x_2 + (x_2 - 3)^2$	(2, 2)	(1, 3)	Local (global) minimum